Tupleدر پایتون

() Tupleیکی از انواع داده‌های مهم در پایتون است که برای نگهداری چند مقدار در یک متغیر استفاده می‌شود

در پایتون چهار نوع داده‌ی مجموعه‌ای (Collection Data Types) اصلی وجود دارد:

نوع داده	مرتب(Ordered)	قابل تغییر(Mutable)	تکراری	نحوه نوشتن
List	yes	yes	yes	[ ]
Tuple	yes	×	yes	( )
Set	×	×*	×	{ }
Dictionary	yes	yes	×برایKey	{key: value}

نکته Tuple :پس از ایجاد قابل تغییر نیست .یعنی نمی‌توانیم عناصر آن را تغییر دهیم، اضافه کنیم یا حذف کنیم

۱- ساختTuple :

برای ساخت Tuple معمولاً از **پرانتز ()**استفاده می‌کنیم

```
fruits = ("apple", "banana", "cherry")
print(fruits)
```

خروجی:

```
('apple', 'banana', 'cherry')
```

Tupleبدون پرانتز :

در پایتون حتی می‌توان Tuple را بدون پرانتز ایجاد کرد. چیزی که در این حالت Tuple را ایجاد می‌کند، comma (,)یا ویرگول است

```
fruits = "apple", "banana", "cherry"
print(fruits)
```

خروجی:

```
('apple', 'banana', 'cherry')
```

بنابراین این دو روش معادل هستند:

```
fruits1 = ("apple", "banana", "cherry")
fruits2 = "apple", "banana", "cherry"
```

۲- ویژگی‌هایTuple :

Tupleسه ویژگی مهم دارد:

۱ — Ordered مرتب هستند :

عناصر Tuple دارای ترتیب مشخص هستند

```
fruits = ("apple", "banana", "cherry")
print(fruits[0])
print(fruits[1])
print(fruits[2])
```

خروجی:

```
apple
banana
cherry
```

هر عنصر دارای یک **Index**است:

apple → index 0

banana → index 1

cherry → index 2

۲- — Unchangeable غیرقابل تغییر هستند :

بعد از ساخته شدنTuple ، نمی‌توانیم مقدار عناصر آن را تغییر دهیم

مثلاً:

```
fruits = ("apple", "banana", "cherry")
fruits[0] = "orange"
```

این کد باعث خطا می‌شود، زیرا Tuple قابل تغییر نیست و TypeError می دهد و این یکی از مهم‌ترین تفاوت‌های Tuple و Listاست

در List:

```
fruits = ["apple", "banana", "cherry"]
fruits[0] = "orange"
print(fruits)
```

خروجی:

```
['orange', 'banana', 'cherry']
```

اما در Tuple:

```
fruits = ("apple", "banana", "cherry")
fruits[0] = "orange"
```

امکان تغییر وجود ندارد

۳ — Allow Duplicates اجازه مقادیر تکراری :

Tupleمی‌تواند چند مقدار یکسان داشته باشد

```
fruits = ("apple", "banana", "cherry", "apple", "cherry")
print(fruits)
```

خروجی:

```
('apple', 'banana', 'cherry', 'apple', 'cherry')
```

بنابراین برخلافSet ، Tupleاجازه‌ی مقادیر تکراری را می‌دهد

۳- دسترسی به عناصر Tuple :

برای دسترسی به عناصر Tuple از Indexاستفاده می‌کنیم

```
colors = ("red", "green", "blue")
print(colors[0])
print(colors[1])
print(colors[2])
```

خروجی:

```
red
green
blue
```

همانندList ، Indexاز **0**شروع می‌شود

Indexمنفی :

می‌توانیم از Index منفی نیز استفاده کنیم

```
colors = ("red", "green", "blue")
print(colors[-1])
print(colors[-2])
print(colors[-3])
```

خروجی:

```
blue
green
red
```

بنابراین:

-1 → آخرین عنصر

-2 → یکی مانده به آخر

-3 → دو تا مانده به آخر

۴- تعداد عناصر Tuple با()len :

برای فهمیدن تعداد عناصر Tuple از تابع ()lenاستفاده می‌کنیم

```
fruits = ("apple", "banana", "cherry")
print(len(fruits))
```

خروجی:

3

مثال دیگر:

```
numbers = (10, 20, 30, 40, 50)
print(len(numbers))
```

خروجی:

5

۵- Tuple با یک عنصر :

اگر بخواهیمTuple ای با فقط یک عنصر ایجاد کنیم، باید بعد از آن عنصر یک commaقرار دهیم

مثلاً:

```
fruit = ("apple",)
print(type(fruit))
```

خروجی:

```
<class 'tuple'>
```

اما اگر comma را ننویسیم:

```
fruit = ("apple")
print(type(fruit))
```

خروجی:

```
<class 'str'>
```

زیرا: ("apple") در واقع فقط یک String است و پرانتزها صرفاً برای گروه‌بندی استفاده شده‌اند

اما("apple",) : یک Tuple است

نکته بسیار مهم

در Tuple تک‌عنصری، ویرگول مهم‌تر از پرانتز است

```
x = ("apple")     # String
y = ("apple",)    # Tuple
```

۶- ساخت Tuple خالی

برای ایجاد یک Tuple خالی کافی است از پرانتزهای خالی استفاده کنیم:

```
my_tuple = ()
print(type(my_tuple))
```

خروجی:

```
<class 'tuple'>
```

۷- Tuple و انواع داده‌ها :

Tupleمی‌تواند شامل انواع مختلف داده باشد

مثلاً:

```
tuple1 = ("apple", "banana", "cherry")
tuple2 = (1, 5, 7, 9, 3)
tuple3 = (True, False, False)
```

همچنین می‌توانیم انواع مختلف داده را در یک Tuple قرار دهیم:

```
person = ("Ali", 43, True, 3.19)
```

در این مثال Tuple شامل موارد زیر است :

"Ali" → String

43 → Integer

True → Boolean

3.19 → Float

۸- بررسی نوع Tuple با()type

برای بررسی نوع یک متغیر می‌توانیم از ()typeاستفاده کنیم

```
fruits = ("apple", "banana", "cherry")
print(type(fruits))
```

خروجی:

```
<class 'tuple'>
```

بنابراین پایتون این متغیر را از نوع tupleتشخیص می‌دهد

۹- ساخت Tuple با()tuple

روش دیگری برای ساخت Tuple استفاده از تابع ()tupleاست

```
fruits = tuple(("apple", "banana", "cherry"))
print(fruits)
```

خروجی:

```
('apple', 'banana', 'cherry')
```

در این روش ممکن است در ابتدا دو پرانتز ببینید:

tuple(("apple", "banana", "cherry"))

پرانتز داخلی مربوط به داده‌هایی است که قرار است به Tuple تبدیل شوند و پرانتز خارجی مربوط به تابع ()tupleاست

۱۰- تبدیل List به Tuple :

یکی از کاربردهای مهم ()tupleتبدیل List به Tuple است

```
fruits_list = ["apple", "banana", "cherry"]
fruits_tuple = tuple(fruits_list)
print(fruits_tuple)
```

خروجی:

```
('apple', 'banana', 'cherry')
```

۱۱- تفاوت اصلی List وTuple

یکی از مهم‌ترین نکات برای یادگیریPython ، تفاوت List و Tuple است

List :

```
fruits = ["apple", "banana", "cherry"]
fruits[0] = "orange"
print(fruits)
```

خروجی:

```
['orange', 'banana', 'cherry']
```

Listقابل تغییر **(Mutable)** است

Tuple :

```
fruits = ("apple", "banana", "cherry")
# fruits[0] = "orange"
```

Tupleقابل تغییر نیست.**(Immutable)**

۱۲- چه زمانی از Tuple استفاده کنیم؟

Tupleزمانی بسیار مفید است که بخواهیم مجموعه‌ای از اطلاعات را نگهداری کنیم که نباید تغییر کنند

مثلاً اطلاعات مختصات یک نقطه:

```
point = (10, 20)
```

یا اطلاعات یک دانشجو:

```
student = ("Ali", 25, "Computer Engineering")
```

یا مشخصات یک محصول:

```
product = ("Laptop", 1500, "Dell")
```

در چنین مواردی اگر اطلاعات نباید تغییر کنند، Tupleانتخاب مناسبی است

۱۳- یک مثال ساده و کاربردی

فرض کنید می‌خواهیم اطلاعات یک دانشجو را ذخیره کنیم:

```
student = ("Ali", 25, "Computer Engineering", 3.19)
print("Name:", student[0])
print("Age:", student[1])
print("Major:", student[2])
print("GPA:", student[3])
```

خروجی:

```
Name: Ali
Age: 25
Major: Computer Engineering
GPA: 3.19
```

در این مثال اطلاعات دانشجو در یک Tuple ذخیره شده است

جمع‌بندی Tuple

Tuple یک Collection در پایتون است که:

- با () ایجاد می‌شود
- Ordered است
- Immutable / Unchangeable است
- اجازه‌ی مقادیر تکراری را می‌دهد
- دارای Index است
- Index از 0 شروع می‌شود
- می‌تواند انواع مختلف داده را نگهداری کند
- برای پیدا کردن تعداد عناصر از len() استفاده می‌کنیم
- برای بررسی نوع از type() استفاده می‌کنیم
- برای ایجاد Tuple با یک عنصر باید حتماً , قرار دهیم
- با tuple() می‌توان داده‌های دیگر مانند List را به Tuple تبدیل کرد

نکات مهم :

```
# List
my_list = ["apple", "banana", "cherry"]

# Tuple
my_tuple = ("apple", "banana", "cherry")
```

تفاوت اصلی:

قابل تغییر → List

غیرقابل تغییر → Tuple

پس اگر داده‌ها ممکن است تغییر کنند، معمولاً List مناسب‌تر است؛ و اگر داده‌ها نباید تغییر کنند، Tuple می‌تواند انتخاب مناسبی باشد

دسترسی به عناصر Tuple در پایتون

در پایتون می‌توانیم به عناصر یک Tuple با استفاده از Indexدسترسی پیدا کنیم. برای مشخص کردنIndex ، از علامت‌های [] استفاده می‌کنیم.

۱- دسترسی به یک عنصر باIndex

هر عنصر در Tuple یک شماره‌ی Index دارد .اولین عنصر از 0شروع می‌شود

```
fruits = ("apple", "banana", "cherry")

print(fruits[0])

print(fruits[1])

print(fruits[2])
```

خروجی:

```
apple

banana

cherry
```

بنابراین:

apple → index 0

banana → index 1

cherry → index 2

اگر بخواهیم عنصر دوم را چاپ کنیم:

```
fruits = ("apple", "banana", "cherry")

print(fruits[1])
```

خروجی:

```
banana
```

نکته :در پایتون Index از 0شروع می‌شود، نه از ۱

۲-Negative Indexing

در پایتون می‌توانیم از Indexمنفی نیز استفاده کنیم

Indexمنفی از انتهای Tuple شروع می‌شود

-1 → آخرین عنصر

-2 → عنصر یکی مانده به آخر

-3 → عنصر دو تا مانده به آخر

مثال:

```
fruits = ("apple", "banana", "cherry")
print(fruits[-1])
```

خروجی:

```
cherry
```

مثال دیگر:

```
fruits = ("apple", "banana", "cherry")
print(fruits[-2])
print(fruits[-3])
```

خروجی:

```
banana
apple
```

بنابراین می‌توانیم عناصر Tuple را هم از ابتدا و هم از انتها پیدا کنیم

۳ — Range of Indexes محدوده‌ای از عناصر

گاهی اوقات نمی‌خواهیم فقط یک عنصر را انتخاب کنیم؛ بلکه می‌خواهیم چند عنصر پشت سر هم را انتخاب کنیم

برای این کار از Slicingاستفاده می‌کنیم

ساختار کلی:

```
tuple[start:end]
```

نکته بسیار مهم این است که:

start → شامل می‌شود

end → شامل نمی‌شود

مثلاً:

```
fruits = ("apple", "banana", "cherry", "orange", "kiwi", "melon", "mango")
print(fruits[2:5])
```

خروجی:

```
('cherry', 'orange', 'kiwi')
```

Index های 2، 3و 4انتخاب می‌شوند و Index 5 شامل نمی‌شود

index 2 → cherry

index 3 → orange

index 4 → kiwi

index 5 → melon ← انتخاب نمی‌شود

بنابراین:

```
fruits[2:5]
```

یعنی:

از Index شماره ۲ شروع کن و قبل از Index شماره ۵ متوقف شو

۴- حذف Start Index

در Slicing لازم نیست همیشه مقدار شروع را مشخص کنیم

اگر مقدار startرا ننویسیم، Pythonاز ابتدای Tuple شروع می‌کند

مثال:

```
fruits = ("apple", "banana", "cherry", "orange", "kiwi", "melon", "mango")
print(fruits[:4])
```

خروجی:

```
('apple', 'banana', 'cherry', 'orange')
```

در اینجا:

```
fruits[:4]
```

یعنی:

از ابتدای Tuple شروع کن و تا قبل از Index شماره ۴ ادامه بده

بنابراینIndex های زیر انتخاب می‌شوند:

0 → apple

1 → banana

2 → cherry

3 → orange

اما Index شماره 4انتخاب نمی‌شود

۵-حذفEnd Index

همچنین می‌توانیم مقدار endرا مشخص نکنیم

در این حالت Python از Index شروع‌شده تا انتهای Tuple ادامه می‌دهد

مثال:

```
fruits = ("apple", "banana", "cherry", "orange", "kiwi", "melon", "mango")
print(fruits[2:])
```

خروجی:

```
('cherry', 'orange', 'kiwi', 'melon', 'mango')
```

یعنی:

از Index شماره ۲ شروع کن و تا انتهای Tuple ادامه بده

۶- Range با Index منفی

می‌توانیم در Slicing ازIndex های منفی نیز استفاده کنیم

مثال:

```
fruits = ("apple", "banana", "cherry", "orange", "kiwi", "melon", "mango")
print(fruits[-4:-1])
```

خروجی:

```
('orange', 'kiwi', 'melon')
```

Indexهای منفی:

apple → -7

banana → -6

cherry → -5

orange → -4

kiwi → -3

melon → -2

mango → -1

در عبارت:

fruits[-4:-1]

از 4-شروع می‌کنیم و قبل از 1-متوقف می‌شویم

بنابراین:

-4 → orange ✓

-3 → kiwi ✓

-2 → melon ✓

-1 → mango ×

نتیجه:

('orange', 'kiwi', 'melon')

۷- بررسی وجود یک عنصر درTuple

گاهی می‌خواهیم بدانیم آیا یک مقدار مشخص در Tuple وجود دارد یا خیر

برای این کار از کلمه کلیدی **in**استفاده می‌کنیم

مثال:

```
fruits = ("apple", "banana", "cherry")
if "apple" in fruits:
  print("Yes, apple is in the fruits tuple")
```

خروجی:

Yes, apple is in the fruits tuple

در اینجا:

"apple" in fruits

بررسی می‌کند که آیا "apple"در Tuple وجود دارد یا خیر

اگر مقدار وجود داشته باشد، نتیجه عبارت inبرابر Trueخواهد بود

مثلاً:

```
fruits = ("apple", "banana", "cherry")
print("apple" in fruits)
```

خروجی:

True

اما:

print("orange" in fruits)

خروجی:

False

۸- استفاده از if برای بررسی وجود عنصر

می‌توانیم نتیجه inرا همراه با ifاستفاده کنیم:

```
fruits = ("apple", "banana", "cherry")
if "banana" in fruits:
  print("Banana exists")
else:
  print("Banana does not exist")
```

خروجی:

Banana exists

۹- مثال کامل

در مثال زیر چند مفهوم را هم‌زمان استفاده می‌کنیم:

```
fruits = ("apple", "banana", "cherry", "orange", "kiwi", "melon", "mango")
print(fruits[1])   # Access by index
print(fruits[-1])  # Negative indexing
print(fruits[2:5])   # Range
print(fruits[:4])    # From beginning
print(fruits[2:])    # To the end
print(fruits[-4:-1])   # Negative range
if "apple" in fruits:      # Check if an item exists
  print("Apple exists")
```

خروجی:

banana

mango

('cherry', 'orange', 'kiwi')

('apple', 'banana', 'cherry', 'orange')

('cherry', 'orange', 'kiwi', 'melon', 'mango')

('orange', 'kiwi', 'melon')

Apple exists

لذا برای دسترسی به عناصر Tuple چند روش مهم داریم:

مثال	Syntax	عملیات
fruits[1]	tuple[index]	یک عنصر
fruits[-1]	tuple[-index]	Indexمنفی
fruits[1:4]	tuple[start:end]	محدوده
fruits[:4]	tuple[:end]	از ابتدا
fruits[2:]	tuple[start:]	تا انتها
fruits[-4:-1]	tuple[-start:-end]	محدوده منفی
"apple" in fruits	item in tuple	بررسی وجود

نکته طلایی درباره Slicing

همیشه به این قانون توجه کنید:

tuple[start:end]

start → شامل می‌شود ✓

end → شامل نمی‌شود ✗

مثلاً:

fruits[2:5]

یعنی:

Index 2 → ✓

Index 3 → ✓

Index 4 → ✓

Index 5 → ✗

این قانون در List، Tupleو بسیاری از ساختارهای دیگر پایتون نیز کاربرد دارد

Update Tuplesدر پایتون:

یکی از مهم‌ترین ویژگی‌های Tuple این است که قابل تغییر نیست.(Immutable)

یعنی بعد از ایجاد یکTuple ، نمی‌توانیم:

- مقدار یک عنصر را تغییر دهیم
- عنصر جدیدی اضافه کنیم
- یک عنصر را حذف کنیم

مثال:

```
fruits = ("apple", "banana", "cherry")
fruits[1] = "orange"
```

این کد باعث خطا می‌شود، زیرا Tuple قابل تغییر نیست

اما در پایتون می‌توانیم با استفاده از چند راهکار غیرمستقیم (Workaround) این کارها را انجام دهیم

۱- تغییر مقدار عناصرTuple

فرض کنید Tuple زیر را داریم:

```
fruits = ("apple", "banana", "cherry")
```

اگر بخواهیم مقدار "banana"را به "kiwi"تغییر دهیم، نمی‌توانیم مستقیماً این کار را انجام دهیم:

```
fruits[1] = "kiwi"
```

زیرا Tuple قابل تغییر نیست

راه‌حل

ابتدا Tuple را به List تبدیل می‌کنیم، سپس مقدار موردنظر را تغییر می‌دهیم و در نهایت List را دوباره به Tuple تبدیل می‌کنیم.

```
fruits = ("apple", "banana", "cherry")

# Convert Tuple to List

x = list(fruits)

# Change the value

x[1] = "kiwi"

# Convert List back to Tuple

fruits = tuple(x)

print(fruits)
```

خروجی:

```
('apple', 'kiwi', 'cherry')
```

مراحل کار:

Tuple

↓

List

↓

Change

↓

Tuple

یعنی:

tuple → list → تغییر → tuple

۲- اضافه کردن عنصر به Tuple

Tuple دارای متد append() نیست

مثلاً این کد اشتباه است:

```
fruits = ("apple", "banana", "cherry")

fruits.append("orange")
```

چون Tuple قابل تغییر نیست

برای اضافه کردن عنصر، دو روش مهم داریم

روش اول: تبدیل Tuple به List

در روش اول Tuple را به List تبدیل می‌کنیم، عنصر جدید را با append() اضافه می‌کنیم و سپس List را دوباره به Tuple تبدیل می‌کنیم

```
fruits = ("apple", "banana", "cherry")

# Convert Tuple to List

x = list(fruits)

# Add new item

x.append("orange")

# Convert List back to Tuple

fruits = tuple(x)

print(fruits)
```

خروجی:

```
('apple', 'banana', 'cherry', 'orange')
```

در این روش:

Tuple

↓

List

↓

append()

↓

Tuple

۳- اضافه کردن یک Tuple به Tuple دیگر

روش دیگری برای اضافه کردن عناصر این است که یک Tuple جدید ایجاد کنیم و آن را با Tuple قبلی ترکیب کنیم.

مثلاً:

```
fruits = ("apple", "banana", "cherry")
new_fruit = ("orange",)
fruits += new_fruit
print(fruits)
```

خروجی:

```
('apple', 'banana', 'cherry', 'orange')
```

در اینجا:

fruits += new_fruit

تقریباً معادل است با:

fruits = fruits + new_fruit

یعنی دو Tuple با یکدیگر ترکیب می‌شوند

نکته مهم در Tuple تک‌عنصری

اگر بخواهیم یک Tuple با فقط یک عنصر ایجاد کنیم، باید بعد از عنصر , commaقرار دهیم

درست:

```
new_fruit = ("orange",)
```

اشتباه:

```
new_fruit = ("orange")
```

در حالت دوم، Pythonآن را Tuple در نظر نمی‌گیرد؛ بلکه یک String است

مثال :

```
x = ("orange",)
print(type(x))
```

خروجی:

```
<class 'tuple'>
```

۴- اضافه کردن چند عنصر بهTuple

می‌توانیم چند عنصر را نیز با ایجاد یک Tuple جدید اضافه کنیم

```
fruits = ("apple", "banana", "cherry")
new_fruits = ("orange", "kiwi")
fruits += new_fruits
print(fruits)
```

خروجی:

```
('apple', 'banana', 'cherry', 'orange', 'kiwi')
```

بنابراین می‌توانیم دو Tuple را با عملگر +به یکدیگر متصل کنیم

```
tuple1 + tuple2
```

مثال:

```
a = (1, 2, 3)
b = (4, 5)
c = a + b
print(c)
```

خروجی:

```
(1, 2, 3, 4, 5)
```

۵- حذف یک عنصر از Tuple

به صورت مستقیم نمی‌توانیم یک عنصر را از Tuple حذف کنیم

مثلاً این کد وجود ندارد:

```
fruits = ("apple", "banana", "cherry")
fruits.remove("apple")
```

زیرا Tuple متد remove() ندارد و قابل تغییر نیست

راه‌حل

مانند تغییر و اضافه کردن، می‌توانیم Tuple را به List تبدیل کنیم

```
fruits = ("apple", "banana", "cherry")
# Convert Tuple to List
x = list(fruits)
# Remove item
x.remove("apple")
# Convert List back to Tuple
fruits = tuple(x)
print(fruits)
```

خروجی:

```
('banana', 'cherry')
```

مراحل:

Tuple

↓

List

↓

remove()

↓

Tuple

۶- حذف کامل Tuple باdel

اگر بخواهیم خود Tuple را به طور کامل از برنامه حذف کنیم، می‌توانیم از دستور delاستفاده کنیم

```
fruits = ("apple", "banana", "cherry")

del fruits
```

بعد از اجرای del، متغیر fruitsدیگر وجود ندارد

اگر بعد از آن بنویسیم:

```
print(fruits)
```

Pythonخطا می‌دهد یعنی در ترمینال خطا و عبارت NameError ظاهر می شود زیرا متغیر fruitsحذف شده است

بنابراین باید تفاوت این دو را بدانیم:

remove an item یعنی حذف یک عنصر ازTuple و del tuple یعنی حذف کامل Tuple

۷- تفاوت remove()و del

این دو دستور کاربرد متفاوتی دارند

remove()

یک عنصر را حذف می‌کند، اما برای Tuple باید ابتدا آن را به List تبدیل کنیم

```
x = list(fruits)

x.remove("apple")

fruits = tuple(x)
```

del

می‌تواند خود متغیر Tuple را به طور کامل حذف کند:

```
del fruits
```

۸- یک مثال کامل

در مثال زیر تغییر، اضافه کردن و حذف را مشاهده می‌کنیم:

```
# Original Tuple
fruits = ("apple", "banana", "cherry")
x = list(fruits)     # Change an item
x[1] = "kiwi"
fruits = tuple(x)
print(fruits)
x = list(fruits)     # Add an item
x.append("orange")
fruits = tuple(x)
print(fruits)
x = list(fruits)      # Remove an item
x.remove("apple")
fruits = tuple(x)
print(fruits)
```

خروجی:

```
('apple', 'kiwi', 'cherry')
('apple', 'kiwi', 'cherry', 'orange')
('kiwi', 'cherry', 'orange')
```

نکته بسیار مهم

اگر مرتباً نیاز دارید عناصر یک مجموعه را اضافه، حذف یا تغییر دهید، بهتر است از Listاستفاده کنید

```
fruits = ["apple", "banana", "cherry"]

fruits[1] = "kiwi"

fruits.append("orange")

fruits.remove("apple")
```

اما اگر اطلاعات شما نباید تغییر کنند، استفاده از Tuple مناسب‌تر است:

```
fruits = ("apple", "banana", "cherry")
```

:بنابراین

List

→ Mutable

→ قابل تغییر

Tuple

→ Immutable

→ غیرقابل تغییر

خلاصه روش‌های Update Tuple

روش جایگزین	امکان مستقیم	عملیات
→ Tuple → List → Tupleتغییر →	✗	تغییر عنصر
Tuple → List → append() → Tuple	✗	اضافه کردن عنصر
استفاده از +یا =+	✓	اضافه کردنTuple
Tuple → List → remove() → Tuple	✗	حذف عنصر
استفاده از del	✓	حذف کلTuple

نکته کلیدی :وقتی می‌گوییم Tuple Immutable است، یعنی خود Tuple پس از ایجاد قابل تغییر نیست. تبدیل آن به List و ساختن یک Tuple جدید، در واقع تغییر Tuple قبلی نیست؛ بلکه یک ساختار جدید ایجاد می‌کنیم

Unpacking Tupleدر پایتون

در پایتون می‌توانیم مقادیر موجود در یک Tuple را مستقیماً به چند متغیر اختصاص دهیم. به این کار Unpackingیا «باز کردن «Tuple گفته می‌شود

قبل از یادگیریUnpacking ، بهتر است با مفهوم Packingآشنا شویم

Packing a Tuple-۱

وقتی چند مقدار را در یک Tuple قرار می‌دهیم، به این کار Packingگفته می‌شود

مثلاً:

```
fruits = ("apple", "banana", "cherry")
```

در اینجا سه مقدار:

apple

banana

cherry

در یک Tuple به نام fruitsقرار گرفته‌اند

به این عمل Packing a Tupleمی‌گوییم

Unpacking a Tuple -۲

در مقابل، می‌توانیم مقادیر داخل Tuple را دوباره خارج کرده و در متغیرهای جداگانه قرار دهیم

به این کار Unpacking a Tupleگفته می‌شود

مثال:

```
fruits = ("apple", "banana", "cherry")
(green, yellow, red) = fruits
print(green)
print(yellow)
print(red)
```

خروجی:

apple

banana

cherry

در این مثال:

green → apple

yellow → banana

red → cherry

یعنی Python مقدار اول را در green، مقدار دوم را در yellowو مقدار سوم را در redقرار می‌دهد

۳- تعداد متغیرها باید با تعداد مقادیر برابر باشد

در حالت معمول، تعداد متغیرهایی که هنگام Unpacking استفاده می‌کنیم باید با تعداد عناصر Tuple برابر باشد

مثلاً:

```
fruits = ("apple", "banana", "cherry")
(a, b, c) = fruits
```

این کد صحیح است، زیرا:

3 values

3 variables

اما اگر تعداد متغیرها کمتر باشد:

```
fruits = ("apple", "banana", "cherry")
(a, b) = fruits
```

Pythonخطا می‌دهد، زیرا سه مقدار وجود دارد ولی فقط دو متغیر تعریف شده است

همچنین اگر تعداد متغیرها بیشتر باشد:

```
fruits = ("apple", "banana", "cherry")
(a, b, c, d) = fruits
```

باز هم خطا ایجاد می‌شود

بنابراین در حالت عادی:

Number of values = Number of variables

۴- استفاده از* Asterisk

گاهی اوقات تعداد مقادیر Tuple بیشتر از تعداد متغیرهایی است که می‌خواهیم تعریف کنیم

در این شرایط می‌توانیم از علامت * asteriskاستفاده کنیم ، علامت *باعث می‌شود Python مقادیر باقی‌مانده را در یک List قرار دهد

مثال:

```
fruits = ("apple", "banana", "cherry", "strawberry", "raspberry")

(green, yellow, *red) = fruits

print(green)

print(yellow)

print(red)
```

خروجی:

```
apple

banana

['cherry', 'strawberry', 'raspberry']
```

در این مثال:

green → apple

yellow → banana

red → ['cherry', 'strawberry', 'raspberry']

نکته مهم این است که متغیر دارای *، مقادیر باقی‌مانده را به صورت Listدریافت می‌کند

یعنی:

red* باعث می‌شود redیک List باشد:

['cherry', 'strawberry', 'raspberry']

۵- استفاده از *در وسط متغیرها

علامت *الزاماً نباید برای آخرین متغیر استفاده شود

می‌توانیم آن را در وسط نیز قرار دهیم

مثال:

```
fruits = ("apple", "mango", "papaya", "pineapple", "cherry")

(green, *tropic, red) = fruits

print(green)

print(tropic)

print(red)
```

خروجی:

```
apple

['mango', 'papaya', 'pineapple']

cherry
```

در اینجا Python به شکل زیر عمل می‌کند:

green → اولین مقدار

red → آخرین مقدار

tropic → تمام مقادیر بین این دو

بنابراین:

green

↓

apple

tropic

↓

mango

papaya

pineapple

red

↓

cherry

متغیر tropicبه صورت یک List ساخته می‌شود:

['mango', 'papaya', 'pineapple']

۶- چرا Python می‌داند چند مقدار را به *اختصاص دهد؟

Pythonسعی می‌کند ابتدا مقادیری را که به متغیرهای معمولی مربوط هستند مشخص کند و سپس تمام مقادیر باقی‌مانده را به متغیری که دارای *است اختصاص دهد

مثلاً:

```
numbers = (1, 2, 3, 4, 5)
(a, *b, c) = numbers
print(a)
print(b)
print(c)
```

خروجی:

```
1
[2, 3, 4]
5
```

Pythonمی‌داند:

a → اولین مقدار

c → آخرین مقدار

b → مقادیر باقی‌مانده

۷- مثال دیگر با *

```
numbers = (10, 20, 30, 40, 50, 60)

(first, second, *remaining) = numbers

print(first)

print(second)

print(remaining)
```

خروجی:

```
10

20

[30, 40, 50, 60]
```

در اینجا:

first → 10

second → 20

remaining → [30, 40, 50, 60]

۸- Unpacking بدون پرانتز

در Python هنگام Unpacking نیازی نیست حتماً از پرانتز استفاده کنیم

مثلاً:

```
fruits = ("apple", "banana", "cherry")

green, yellow, red = fruits

print(green)

print(yellow)

print(red)
```

این کد کاملاً صحیح است

یعنی این دو روش یکسان هستند:

(green, yellow, red) = fruits

green, yellow, red = fruits

۹- کاربرد Unpacking در برنامه‌نویسی

Unpackingدر برنامه‌نویسی بسیار کاربردی است مثلاً فرض کنید مختصات یک نقطه را داریم:

```
point = (10, 20)
x, y = point
print("X:", x)
print("Y:", y)
```

خروجی:

X: 10

Y: 20

در اینجا:

point

↓

(10, 20)

x → 10

y → 20

۱۰- Unpacking برای اطلاعات یک دانشجو

مثال دیگری از کاربرد:Tuple

```
student = ("Ali", 25, "Computer Engineering")
name, age, major = student
print("Name:", name)
print("Age:", age)
print("Major:", major)
```

خروجی:

```
Name: Ali
Age: 25
Major: Computer Engineering
```

این روش باعث می‌شود دسترسی به اطلاعات Tuple ساده‌تر و خواناتر شود

بنابراین در Python دو مفهوم مهم داریم:

Packing : یعنی قرار دادن چند مقدار در یک:Tuple

```
fruits = ("apple", "banana", "cherry")
```

Unpacking : یعنی خارج کردن مقادیر Tuple و قرار دادن آن‌ها در متغیرهای مختلف:

```
green, yellow, red = fruits
```

استفاده از *

اگر تعداد مقادیر بیشتر از تعداد متغیرها باشد، می‌توانیم از *استفاده کنیم:

```
fruits = ("apple", "banana", "cherry", "orange")
first, *remaining = fruits
print(first)
print(remaining)
```

خروجی:

apple

['banana', 'cherry', 'orange']

نکته بسیار مهم:

Unpackingمعمولی :تعداد متغیرها = تعداد مقادیر

Unpackingبا:*

متغیر دارای * ⟵ مقادیر باقی‌مانده را به صورت List دریافت می‌کند

بنابراین *ابزار بسیار مفیدی برای جمع‌آوری چند مقدار باقی‌مانده هنگام Unpacking

Loop Through Tupleدر پایتون

گاهی اوقات لازم است تمام عناصر یک Tuple را یکی‌یکی بررسی کنیم یا روی همه‌ی عناصر آن یک عملیات انجام دهیم. برای این کار می‌توانیم از حلقه‌های forو whileاستفاده کنیم

سه روش مهم برای پیمایش (Loop) روی Tuple عبارت‌اند از:

۱. استفاده از forبه صورت مستقیم

۲. استفاده از forهمراه باIndex

۳. استفاده از whileهمراه باIndex

۱- استفاده از حلقه for

ساده‌ترین روش برای پیمایش عناصر Tuple استفاده از حلقه forاست

```
fruits = ("apple", "banana", "cherry")
for fruit in fruits:
  print(fruit)
```

خروجی:

apple

banana

cherry

در اینجا Python هر بار یکی از عناصر Tuple را در متغیر fruitقرار می‌دهد

روند اجرای برنامه:

دور اول → fruit = "apple"

دور دوم → fruit = "banana"

دور سوم → fruit = "cherry"

بنابراین نیازی نیست خودمانIndex ها را مدیریت کنیم

۲- پیمایش Tuple باIndex

روش دیگر این است که به جای دسترسی مستقیم به عناصر، از **Index**استفاده کنیم

برای این کار معمولاً از دو تابع ()lenو ()rangeاستفاده می‌کنیم

```
fruits = ("apple", "banana", "cherry")
for i in range(len(fruits)):
   print(fruits[i])
```

خروجی:

apple

banana

cherry

این کد را بررسی می کنیم :

()len

تابع ()lenتعداد عناصر Tuple را مشخص می‌کند:

len(fruits)

نتیجه: ۳

()range

بنابراین:

range(len(fruits))

تبدیل می‌شود به:

range(3)

که**Index** های زیر را ایجاد می‌کند:

0

1

2

سپس در هر مرحله:

fruits[i]

عنصر مربوط به آن Index را برمی‌گرداند

روند اجرا:

i = 0 → fruits[0] → apple

i = 1 → fruits[1] → banana

i = 2 → fruits[2] → cherry

۳- چرا از range(len())استفاده می‌کنیم؟

اگر بخواهیم روی**Index** های Tuple حرکت کنیم، باید بدانیم Tuple چند عنصر دارد

مثلاً:

```
fruits = ("apple", "banana", "cherry")
print(len(fruits))
```

خروجی:

3

پس**Index** های معتبر عبارت‌اند از:

0

1

2

به همین دلیل می‌توانیم بنویسیم:

```
for i in range(len(fruits)):
  print(fruits[i])
```

۴- استفاده از while

روش دیگری برای پیمایش Tuple استفاده از حلقه whileاست

در این روش باید خودمان Index را مدیریت کنیم

```
fruits = ("apple", "banana", "cherry")
i = 0
while i < len(fruits):
  print(fruits[i])
  i = i + 1
```

خروجی:

```
apple
banana
cherry
```

در اینجا ابتدا:

```
i = 0
```

قرار می‌دهیم، زیرا اولین Index برابر 0است

سپس شرط را بررسی می‌کنیم:

```
i < len(fruits)
```

چون:

```
0 < 3
```

شرط درست است و حلقه اجرا می‌شود

بعد:

i = i + 1

اجرا می‌شود و مقدار iبه 1افزایش پیدا می‌کند

روند اجرای برنامه:

i = 0 → fruits[0] → apple

i = 1 → fruits[1] → banana

i = 2 → fruits[2] → cherry

i = 3 → شرط ۳ 3 < → پایان حلقه ⟵ نادرست

۵-مقایسه سه روش

فرض کنید Tuple زیر را داریم:

```
fruits = ("apple", "banana", "cherry")
```

روش اول for :مستقیم

```
for fruit in fruits:
    print(fruit)
```

روش دوم for :باIndex

```
for i in range(len(fruits)):
    print(fruits[i])
```

روش سومwhile :

```
i = 0
while i < len(fruits):
    print(fruits[i])
    i += 1
```

هر سه روش نتیجه یکسانی تولید می‌کنند:

apple

banana

cherry

۶-کدام روش بهتر است؟

برای یک پیمایش ساده، معمولاً روش اول با for بهترین و ساده‌ترین انتخاب است:

```
fruits = ("apple", "banana", "cherry")
for fruit in fruits:
  print(fruit)
```

زیرا کد کوتاه‌تر و خواناتر است

اما زمانی که به Index نیاز داریم، استفاده از range(len()) می‌تواند مفید باشد

مثلاً:

```
fruits = ("apple", "banana", "cherry")
for i in range(len(fruits)):
  print("Index:", i)
  print("Value:", fruits[i])
```

خروجی:

Index: 0

Value: apple

Index: 1

Value: banana

Index: 2

Value: cherry

۷- یک مثال کاربردی

فرض کنید Tuple شامل نمرات دانش‌آموزان باشد:

```
scores = (18, 15, 20, 17, 19)
for score in scores:
   print("Score:", score)
```

خروجی:

```
Score: 18
Score: 15
Score: 20
Score: 17
Score: 19
```

همچنین می‌توانیم با استفاده از if روی هر عنصر عملیات انجام دهیم:

```
scores = (18, 15, 20, 17, 19)
for score in scores:
   if score >= 18:
      print(score, "Excellent")
```

خروجی:

```
18 Excellent
20 Excellent
19 Excellent
```

۸-مثال باIndex

فرض کنید بخواهیم علاوه بر مقدار، شماره Index را نیز نمایش دهیم:

```
fruits = ("apple", "banana", "cherry")
for i in range(len(fruits)):
  print("Index:", i, "Fruit:", fruits[i])
```

خروجی:

```
Index: 0 Fruit: apple
Index: 1 Fruit: banana
Index: 2 Fruit: cherry
```

جمع‌بندی

برای Loop کردن روی Tuple می‌توانیم از روش‌های مختلفی استفاده کنیم

روش اول for — مستقیم

```
for item in my_tuple:
  print(item)
```

این روش ساده‌ترین و رایج‌ترین روش است

روش دوم for — باIndex

```
for i in range(len(my_tuple)):
  print(my_tuple[i])
```

زمانی مناسب است که به Indexو مقدار نیاز داشته باشیم

روش سومwhile —

```
i = 0
while i < len(my_tuple):
  print(my_tuple[i])
```

```
    i += 1
```

در این روش باید Index را خودمان کنترل کنیم

نکته مهم

for → ساده‌تر و خواناتر

for + range() → مناسب برای کار با Index

while → مناسب برای زمانی که کنترل بیشتری روی شرط و روند حلقه نیاز داریم

در برنامه‌های معمولPython ، اگر فقط می‌خواهیم تمام عناصر Tuple را یکی‌یکی بخوانیم، معمولاً این روش پیشنهاد می‌شود:

```
for item in my_tuple:
    print(item)
```

Join Tuplesدر پایتون :

در پایتون می‌توانیم چند Tuple را با یکدیگر ترکیب کنیم یا محتوای یک Tuple را چند بار تکرار کنیم

برای این کار از دو عملگر مهم استفاده می‌کنیم:

- +برای ترکیبTuple (Join) ها
- *برای تکرارTuple (Multiply)

۱-Join Two Tuples

برای ترکیب دو یا چند Tuple می‌توانیم از عملگر +استفاده کنیم

مثال:

```
tuple1 = ("a", "b", "c")
tuple2 = (1, 2, 3)
tuple3 = tuple1 + tuple2
print(tuple3)
```

خروجی:

```
('a', 'b', 'c', 1, 2, 3)
```

در اینجا:

tuple1 + tuple2

دو Tuple را پشت سر هم قرار می‌دهد.

به صورت تصویری:

tuple1

("a", "b", "c")

+

tuple2

(1, 2, 3)

↓

tuple3

("a", "b", "c", 1, 2, 3)

۲- ترکیب چندTuple

می‌توانیم بیش از دو Tuple را نیز با هم ترکیب کنیم

```
tuple1 = (1, 2)
tuple2 = (3, 4)
tuple3 = (5, 6)
result = tuple1 + tuple2 + tuple3
print(result)
```

خروجی:

(1, 2, 3, 4, 5, 6)

همچنین می‌توانیم این کار را به صورت مرحله‌ای انجام دهیم:

```
tuple1 = (1, 2)
tuple2 = (3, 4)
```

```
result = tuple1 + tuple2
print(result)
```

۳- نکته مهم درباره + : عملگر +برای ترکیب دو Tuple استفاده می‌شود و هر دو طرف عملگر باید Tuple باشند

مثلاً:

```
tuple1 = ("apple", "banana")
tuple2 = ("cherry", "orange")
result = tuple1 + tuple2
print(result)
```

خروجی:

```
('apple', 'banana', 'cherry', 'orange')
```

اما بهتر است بدانیم که Tuple با List یکسان نیست

مثلاً:

```
tuple1 = ("apple", "banana")
my_list = ["cherry", "orange"]
result = tuple1 + my_list
```

این کار باعث خطا می‌شود، زیرا یکی Tuple و دیگری List است

۴-Multiply Tuples

عملگر *برای تکرار کردن عناصر Tuple استفاده می‌شود

مثلاً:

```
fruits = ("apple", "banana", "cherry")
mytuple = fruits * 2
print(mytuple)
```

خروجی:

```
('apple', 'banana', 'cherry', 'apple', 'banana', 'cherry')
```

در اینجا Tuple دو بار تکرار شده است یعنی:

```
("apple", "banana", "cherry")
            × 2
            ↓
("apple", "banana", "cherry",
 "apple", "banana", "cherry")
```

۵- تکرار Tuple با اعداد مختلف

می‌توانیم Tuple را هر تعداد بار که بخواهیم تکرار کنیم

مثلاً:

```
numbers = (1, 2, 3)
print(numbers * 2)
print(numbers * 3)
print(numbers * 4)
```

خروجی:

```
(1, 2, 3, 1, 2, 3)
(1, 2, 3, 1, 2, 3, 1, 2, 3)
(1, 2, 3, 1, 2, 3, 1, 2, 3, 1, 2, 3)
```

بنابراین:

numbers * 3 ، یعنی محتوای Tuple سه بار پشت سر هم تکرار شود

۶- تفاوت +و *

این دو عملگر کاربرد متفاوتی دارند

→ +ترکیبTuple ها

```
a = (1, 2)
b = (3, 4)
result = a + b
print(result)
```

خروجی:

```
(1, 2, 3, 4)
```

Tuple تکرار → *

```
a = (1, 2)
result = a * 3
print(result)
```

خروجی:

```
(1, 2, 1, 2, 1, 2)
```

۷- Tuple خالی و عملگر +

حتی می‌توانیم یک Tuple خالی را با Tuple دیگری ترکیب کنیم:

```
a = ()
b = (1, 2, 3)
result = a + b
```

```
print(result)
```

خروجی:

```
(1, 2, 3)
```

۸- یک مثال کاربردی

فرض کنید دو Tuple شامل نام دانشجویان دو کلاس داریم:

```
class_a = ("Ali", "Sara", "Reza")
class_b = ("Mina", "John", "David")
all_students = class_a + class_b
print(all_students)
```

خروجی:

```
('Ali', 'Sara', 'Reza', 'Mina', 'John', 'David')
```

در اینجا با استفاده از +اطلاعات دو Tuple را با هم ترکیب کرده‌ایم

۹- یک مثال با تکرار

فرض کنید یک Tuple شامل روزهای هفته داریم و می‌خواهیم آن را دو بار تکرار کنیم:

```
days = ("Saturday", "Sunday", "Monday")
new_days = days * 2
print(new_days)
```

خروجی:

```
('Saturday', 'Sunday', 'Monday', 'Saturday', 'Sunday', 'Monday')
```

جمع‌بندی

برای کار با چندTuple ، دو عملگر مهم داریم:

عملیات	عملگر	مثال	نتیجه
ترکیبTuple ها	+	(1,2) + (3,4)	(1,2,3,4)
تکرارTuple	*	(1,2) * 3	(1,2,1,2,1,2)

ترکیبTuple

```
tuple1 = ("a", "b")

tuple2 = ("c", "d")

result = tuple1 + tuple2

print(result)
```

نتیجه:

```
('a', 'b', 'c', 'd')
```

تکرارTuple

```
fruits = ("apple", "banana")

result = fruits * 3

print(result)
```

نتیجه:

```
('apple', 'banana', 'apple', 'banana', 'apple', 'banana')
```

نکته مهم :چون Tuple یک ساختار Immutableاست، عملیات +و Tuple *اصلی را تغییر نمی‌دهند؛ بلکه یک Tupleجدید ایجاد می‌کنند

Tuple Methodsدر پایتون :

Tupleدر پایتون به دلیل Immutableبودن، متدهای بسیار کمتری نسبت به List دارد و در واقع، Tupleبه صورت مستقیم فقط دو متد اصلی دارد:

۱. count()

۲. index()

این دو متد برای شمارش عناصر و پیدا کردن موقعیت یک عنصر استفاده می‌شوند.

۱- متد ()count

متد ()countتعداد دفعاتی را که یک مقدار مشخص در Tuple تکرار شده است، برمی‌گرداند

ساختار کلی:

tuple.count(value)

مثال

```
fruits = ("apple", "banana", "cherry", "apple", "apple")
result = fruits.count("apple")
print(result)
```

خروجی:

3

زیرا "apple"سه بار در Tuple وجود دارد

مثال دیگر

```
numbers = (10, 20, 10, 30, 10, 40)
print(numbers.count(10))
```

خروجی:

3

عدد 10سه بار در Tuple وجود دارد

اما:

```
print(numbers.count(50))
```

خروجی:

0

اگر مقدار موردنظر در Tuple وجود نداشته باشد، ()countمقدار 0را برمی‌گرداند

۲-متد index()

متد index()برای پیدا کردن Indexاولین وقوع یک مقدار در Tuple استفاده می‌شود

ساختار کلی:

tuple.index(value)

مثال:

```
fruits = ("apple", "banana", "cherry")
result = fruits.index("banana")
print(result)
```

خروجی:

1

زیرا "banana"در Index شماره 1قرار دارد

apple → 0

banana → 1

cherry → 2

۳- اگر یک مقدار چند بار تکرار شود

اگر یک مقدار چند بار در Tuple وجود داشته باشد، index()فقط Indexاولین occurrence را برمی‌گرداند

مثلاً:

```
fruits = ("apple", "banana", "apple", "cherry", "apple")
print(fruits.index("apple"))
```

خروجی:

0

با اینکه "apple"درIndex های 0، 2و 4وجود دارد، index()فقط اولین Index یعنی 0را برمی‌گرداند

برای پیدا کردن تعداد تکرارهای آن می‌توانیم از count()استفاده کنیم:

```
print(fruits.count("apple"))
```

خروجی:

```
3
```

۴- اگر مقدار وجود نداشته باشد

اگر مقداری که به index()می‌دهیم در Tuple وجود نداشته باشد، Pythonخطای ValueErrorایجاد می‌کند

مثلاً:

```
fruits = ("apple", "banana", "cherry")
print(fruits.index("orange"))
```

چون "orange"در Tuple وجود ندارد، خطایی مشابه زیر دریافت می‌کنیم:

```
ValueError: tuple.index(x): x not in tuple
```

بنابراین اگر مطمئن نیستیم یک مقدار در Tuple وجود دارد یا خیر، بهتر است ابتدا با inبررسی کنیم:

```
fruits = ("apple", "banana", "cherry")
if "orange" in fruits:
  print(fruits.index("orange"))
else:
  print("Item not found")
```

خروجی:

```
Item not found
```

۵- ترکیب count() و index()

می‌توانیم از هر دو متد در یک برنامه استفاده کنیم

```
numbers = (10, 20, 30, 20, 40, 20)
print("Count:", numbers.count(20))
print("First index:", numbers.index(20))
```

خروجی:

```
Count: 3
First index: 1
```

یعنی:

20 → سه بار تکرار شده است

20 → اولین بار در Index شماره ۱ قرار دارد

6. Tuple فقط دو Method اصلی دارد

برخلاف List که متدهای زیادی مانند:

append()

insert()

remove()

pop()

clear()

sort()

reverse()

دارد، Tuple فقط دو متد اصلی دارد:

count()

index()

دلیل اصلی این موضوع Immutableبودن Tuple است

چون Tuple قابل تغییر نیست، متدهایی مانند ()appendیا ()removeدر آن وجود ندارند

مثلاً این کد اشتباه است:

```
fruits = ("apple", "banana", "cherry")
fruits.append("orange")
```

و Python خطا می‌دهد

همچنین:

```
fruits.remove("apple")
```

نیز برای Tuple وجود ندارد

۷-تفاوت ()countو ()index

تفاوت این دو متد را به خاطر بسپارید:

()count

می‌پرسد:

این مقدار چند بار در Tuple وجود دارد؟

مثال:

```
numbers = (1, 2, 2, 3, 2)
print(numbers.count(2))
```

نتیجه:

3

()index

می‌پرسد:

این مقدار اولین بار در چهIndex ای قرار دارد؟

مثال:

```
numbers = (1, 2, 2, 3, 2)
```

```
print(numbers.index(2))
```

نتیجه:

1

جمع‌بندی Tuple Methods

Method	کاربرد	مثال	نتیجه
count()	تعداد تکرار یک مقدار	t.count("apple")	تعداد تکرار
index()	پیدا کردن اولین Index یک مقدار	t.index("apple")	شماره Index

مثال نهایی

```
fruits = ( "apple", "banana",  "apple", "cherry", "apple")

print(fruits.count("apple"))

print(fruits.index("apple"))
```

خروجی:

3

0

یعنی:

count("apple") → تعداد apple ها = ۳

index("apple") → Index اولین apple در = 0

نکته طلایی

count() → چند بار؟

index() → کجا؟

این دو متد مهم‌ترین متدهای داخلی Tuple هستند و برای شمارش و جستجوی عناصر مورد استفاده قرار می‌گیرند